Design Document: ExplainIt V1

Oct 17, 2026 · @Kaushtubh Mishra

Overview and design principles

Status: final for V1. Updated with the results of the working prototype (see the Prototype Design doc, "Findings so far"). The script, voice, and video pipeline described here already exists in code; the web app, database, worker, and storage sections are what remains to build.

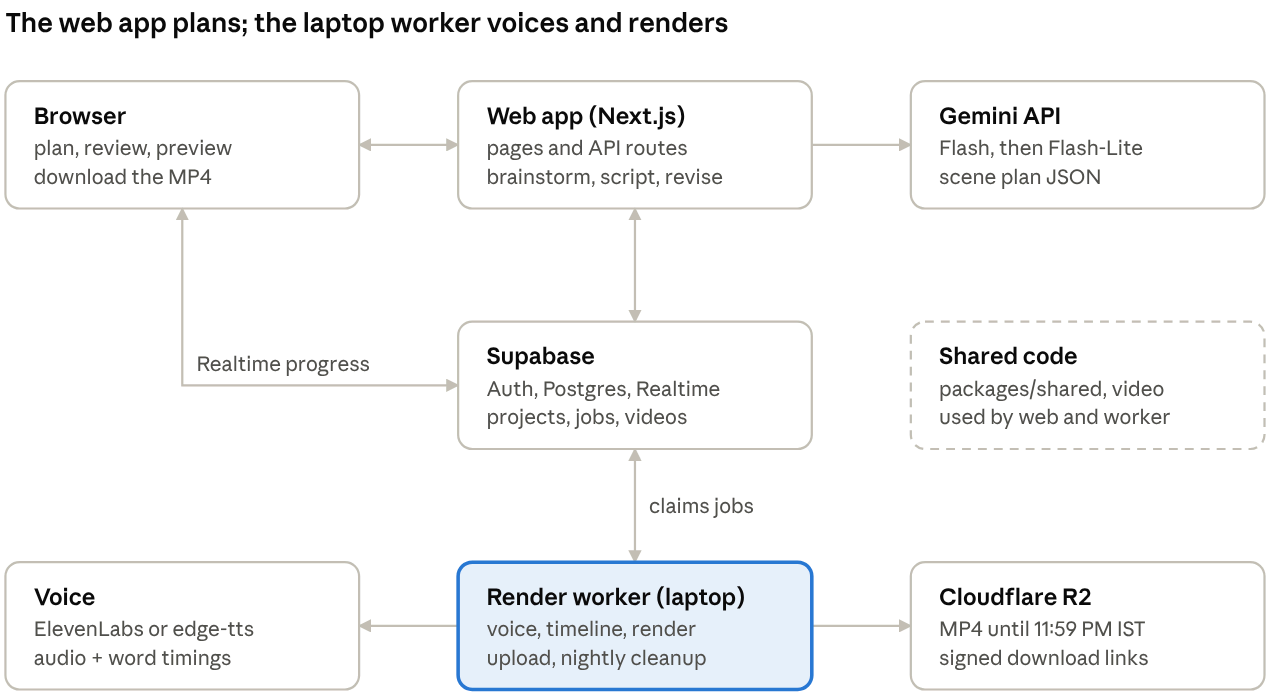
This document defines how ExplainIt V1 is built: one TypeScript codebase on the builder's Windows laptop, a Next.js web app, a local render worker, Supabase for auth and data, and Cloudflare R2 for short-lived video storage. It implements the PRD; where they differ, the PRD wins on *what* and this document wins on *how*.

Six principles guide every decision:

1. **The scene plan is the contract.** One validated JSON format connects the LLM, the review screen, and the renderer. Nothing else passes between them.
2. **Templates, not generated animation.** The LLM fills fields in fixed Remotion templates; it never writes code or free-form visuals.
3. **Every provider sits behind an adapter.** Gemini Flash or Flash-Lite, ElevenLabs or edge-tts, are swapped by config, not by code changes.
4. **Cache by content, re-do only what changed.** Voice audio is stored per scene under a hash of its inputs, so a revision re-voices only the scenes whose words changed. The video is re-rendered in full, which the prototype showed takes only 2–4 minutes.
5. **Every stage is resumable.** Each job stage saves its output before the next starts, so a failure retries from where it stopped.
6. **Zero cost and local-first.** Code stays on the laptop (no GitHub remote); only login, data, and finished videos live in free cloud tiers.

Architecture overview

The web app handles everything interactive and the laptop worker handles everything heavy; Supabase sits between them, so neither ever calls the other directly.



V1 architecture · web app, Supabase, laptop worker, R2

The browser plans and reviews through the web app, which calls Gemini and saves to Supabase; clicking Render creates a job that the worker claims, voices, renders, and uploads to R2, and the browser follows progress live and downloads through a signed link. Both apps import the same shared packages, so the storyboard preview and the final render use identical templates.

Repository structure

The code is one npm workspaces project at `C:\projects\explainit`, with two apps and two shared packages, all in TypeScript. Generated media lives on D: so the SSD holds only code.

```
C:\projects\explainit\
  package.json              workspace root; scripts: plan, voice, render,
make-video, studio, eval
  .env / .env.example       secrets and settings / names only
  apps\                     (to build)
    web\                    Next.js app: pages, API routes, storyboard review
    worker\                 Node render worker: claims jobs, voices, renders,
uploads, cleans up
  packages\
    shared\src\             schema, rules, prompts, providers (LLM, voice),
timeline, generatePlan, voicePlan
    video\src\              Remotion composition, 7 templates, captions,
themes, fonts, icons
    video\public\fonts\     Poppins and Noto Sans Devanagari (bundled,
offline)
  scripts\                  command-line pipeline (generate-plan, voice-plan,
render-video, make-video,
                            compare-voices, eval-prompts) and steps.ts shared
by them
  tools\edge_tts_words.py   Python helper that returns edge-tts audio and
word timings

D:\ExplainIt\
  runs\{date}_{time}_{slug}\   prototype runs: plan, audio, timing, timeline,
video.mp4, reports
  cache\{projectId}\audio\     V1 worker: per-scene MP3 and timing, keyed by
audio hash
  out\{jobId}.mp4              V1 worker: final video before upload
  music\                       background tracks plus their license pages
```

Part	Runs where	Talks to
`apps/web`	Laptop in development; Vercel later (deployed with the Vercel CLI, no GitHub needed)	Supabase, Gemini, R2 (signed download links)
`apps/worker`	Laptop only	Supabase, ElevenLabs, edge-tts, Remotion, R2
`packages/shared`	Imported by both apps	Nothing on its own

Part	Runs where	Talks to
`packages/video`	Imported by the worker; previewed in Remotion Studio	Nothing on its own

Version control is a local Git repository with no remote, so every change can be undone without uploading anything.

Scene plan contract

The scene plan is defined once in `packages/shared` with Zod, a validation library that gives both TypeScript types and runtime checks from the same definition. Every LLM response, every user edit, and every render input passes through `parseScenePlan()` before use.

```
export const Language = z.enum(["en-IN", "hi-IN"]);
export const Theme = z.enum(["sky", "forest", "sunset", "night"]);
export const IconName = z.enum(ICON_NAMES); // about 130 Lucide names:
science, phones, computers

// cue = the narration word at which an item appears
const Item = z.object({ icon: IconName, label: z.string().max(32), cue:
z.string().optional() });
const Point = z.object({ text: z.string().max(40), cue: z.string().optional()
});
const Side = z.object({ title: z.string().max(24), icon: IconName, points:
z.array(Point).max(3) });
const PhoneAction = z.object({
  type: z.enum(["tap", "type", "scroll", "toggle", "wait"]),
  target: z.string().max(28), // exact on-screen text, or the text being
typed
  label: z.string().max(44),  // instruction shown above the phone
  cue: z.string().optional(),
});

export const SceneContent = z.discriminatedUnion("template", [
  z.object({ template: z.literal("TitleCard"), headline: z.string().max(40),
subtitle: z.string().max(60).optional(), icon: IconName }),
  z.object({ template: z.literal("IconScene"), icon: IconName, label:
z.string().max(32), caption: z.string().max(80) }),
  z.object({ template: z.literal("StepFlow"), steps:
```

```
z.array(Item).min(3).max(5) }),
  z.object({ template: z.literal("CycleDiagram"), nodes:
z.array(Item).min(3).max(5) }),
  z.object({ template: z.literal("Comparison"), left: Side, right: Side }),
  z.object({ template: z.literal("Recap"), headline: z.string().max(40),
points: z.array(Point).min(1).max(3) }),
  z.object({ template: z.literal("PhoneScreen"), title: z.string().max(24),
actions: z.array(PhoneAction).min(1).max(4) }),
]);

export const Scene = z.object({
  id: z.string().regex(/^s\d+$/),        // stable across revisions
  narration: z.string().min(5).max(400),
  captionOverride: z.string().optional(), // FR-12: edited caption text
  content: SceneContent,
});

export const ScenePlan = z.object({
  version: z.literal(1), title: z.string().max(70), language: Language,
theme: Theme,
  scenes: z.array(Scene).min(5).max(10),
});
```

Rules checked after the schema, in `checkPlanRules()`:

Rule	Source	Check	On failure
First scene is a hook	FR-4	Scene 1 uses TitleCard or IconScene	Repair request
Last scene is a recap	FR-4	Last scene uses Recap	Repair request
Length fits 60–90 s	FR-3	Narration fits the chosen length: 125–155 words for 60 s, 185–220 words for 90 s, in both languages	Repair request
Scene ids are unique	FR-6	No duplicate ids	Repair request
Text fits its field	Schema	Labels, headlines, and points within their limits	Shortened automatically at a word boundary

Rule	Source	Check	On failure
Scenes stay short	FR-3	Scene narration over 45 words	Warning only
Cues are spoken	Timing	Each cue appears in its scene's narration	Warning; that item uses even timing

The `template` field lives inside `content` so TypeScript knows exactly which fields each template has. The `id` of a scene never changes once created, which is what lets revisions and the cache track scenes.

Template library

Each template is a Remotion component in `packages/video/templates` that takes a scene's content, its precomputed timing (beats and captions), and the theme, and fills a 1080×1920 frame at 30 fps. There are seven templates.

Template	Used for	Layout and animation
TitleCard	The hook	Large icon with a rotating dashed ring, pulsing and bobbing; headline and subtitle rise in
IconScene	One idea	Label on top, a 440 px icon that floats and pulses, caption below
StepFlow	One-way processes	Focus mode: one large step at a time slides in and out; "Step 3 of 7" with progress dots, numbered across back-to-back step scenes
CycleDiagram	Processes that loop	Nodes on a circle pop in at their cues; arrows draw themselves between nodes; a centre icon turns
Comparison	Two different things	Two full-height columns with large icons and a pulsing VS badge; points tick in at their cues
Recap	The ending	Headline, then checklist cards that slide in with a ticking checkmark

Template	Used for	Layout and animation
PhoneScreen	How-tos on a phone	A phone mockup with the current instruction above it; a finger glides to each element and taps (press and ripple), types text letter by letter, scrolls, flips toggles, or waits while a download bar fills

Shared rules for all templates:

- **Safe area:** content sits between 220 px from the top and 460 px from the bottom, where Shorts and Reels draw their buttons. Captions sit just above the bottom line.
- **Never an empty screen:** each scene's first item appears within 0.4 s; cue words time every later item.
- **Constant motion:** icons bob and pulse, rings rotate, every scene slowly pushes in by about 4.5%, and the background gradient drifts.
- **Duration:** a scene lasts `ceil((audioSeconds + 0.3) × 30)` frames plus the 10-frame transition (15 frames for the last scene).
- **Transitions:** Remotion's `TransitionSeries` alternates fade, slide, and wipe between scenes.
- **Themes:** 4 themes (sky, forest, sunset, night) in `themes.ts`, each a gradient, accent, card, and text colours.
- **Fonts:** Poppins and Noto Sans Devanagari bundled as files and loaded with `@remotion/fonts`, so renders work offline and Hindi shapes correctly.
- **Icons:** Lucide icons by name from the allowlist; two renamed icons are mapped by alias in `Icon.tsx`, and unknown names fall back to a sparkle.

Every template gets a sample scene in Remotion Studio (`npm run studio`), so it can be previewed and adjusted without running the full pipeline.

Provider adapters

Three small interfaces in `packages/shared/providers` hide every outside service, so the rest of the code never imports a vendor SDK directly.

```
interface LLMProvider {
  name: string;
  generateText(req: { system: string; messages: Msg[]; json?: boolean }):
Promise<string>;
```

```
}
// generatePlan() validates and repairs the text; providers stay simple.

interface VoiceProvider {
  name: string;
  creditsPerCharacter: number;               // 1 for Multilingual v2, 0.5
for Flash, 0 for edge-tts
  voiceFor(language: Language): string;
  synthesize(req: { text: string; language: Language; voiceId: string;
mp3Path: string }): Promise<VoiceResult>;
  remainingCredits?(): Promise<number | null>;
}
type VoiceResult = { words: WordTiming[]; seconds: number; characters: number
};
type WordTiming = { word: string; start: number; end: number }; // seconds

interface VideoStore {                         // to build in M5
  upload(key: string, file: string): Promise<void>;
  signedDownloadUrl(key: string, minutes: number): Promise<string>;
  remove(keys: string[]): Promise<void>;
}
```

Interface	Primary	Fallback	How fallback is chosen
LLMProvider	Gemini Flash	Gemini Flash-Lite (Ollama only on request, for offline tests)	On rate limit (429) or outage, after waiting 5 s and retrying once. Ollama is not in the automatic chain: on this laptop's CPU it took about 135 s per call. A hosted fallback is chosen at Vercel deployment.
VoiceProvider	ElevenLabs (multilingual model, Indian English or Hindi voice)	edge-tts (en-IN or hi-IN voices)	`TTS_PROVIDER` setting, or automatically when ElevenLabs quota is too low for the job

Interface	Primary	Fallback	How fallback is chosen
VideoStore	Cloudflare R2 through the S3-compatible SDK	Local folder `D:\ExplainIt\out`	Local store used in M1–M4 before R2 is set up

Implementation notes:

- **Gemini:** official `@google/genai` SDK with `responseMimeType: application/json` and `thinkingLevel` set from `GEMINI_THINKING` (default MINIMAL), which cut a script from 263 s to about 30–40 s. If a model rejects the thinking setting, the adapter retries without it.
- **Ollama:** `http://localhost:11434/api/chat` with `format: json` and `keep_alive: 30m`; used only with `--llm ollama`.
- **ElevenLabs:** the `with-timestamps` endpoint at `mp3_44100_128`; character timings are grouped into words, credits are checked against `/v1/user/subscription` before any synthesis, and 429 or 5xx responses retry 3 times.
- **edge-tts:** `tools/edge_tts_words.py` streams audio and word-boundary events; text goes in as UTF-8 JSON on stdin, so Hindi is safe on Windows. `EDGE_RATE` (default +10%) sets the speaking speed.
- **Audio length:** both providers return constant-bitrate MP3, so length is file size × 8 ÷ bitrate, with no extra tools.

The choice of provider and model lives in `.env`, so switching never needs a code change.

Prompt design

Three prompts drive all LLM work, each stored as a versioned text file in `packages/shared/prompts` so changes can be compared over time.

Prompt	Called by	Input	Output
`brainstorm.v1`	Planning screen, brainstorm mode	Chat history	A short reply, or `{ "confirmed": true, "topic": "..." }` once the user agrees on a topic

Prompt	Called by	Input	Output
`script.v1`	"Generate script"	Topic or notes, language, allowed templates and icons	A full ScenePlan
`revise.v1`	Change requests at storyboard or video stage	Current ScenePlan, the request, the stage	`{ "plan": ScenePlan, "changedSceneIds": [...] }`

Core rules in `script.v1`:

1. Educational explainers of the length the user chose (60 s: 125–155 words; 90 s: 185–220 words), 5–10 scenes, each 15–40 words.
2. Scene 1 is a hook question (TitleCard or IconScene); the last scene is a Recap.
3. Spoken text only: short sentences, no lists, symbols, or markdown. Stick to well-established facts.
4. Only the listed templates, fields, and icon names; pick icons that match their own label. Use the theme the user chose; only when it is "auto", pick the theme that suits the topic.
5. Every on-screen label, headline, and point restates something the narration says.
6. CycleDiagram only for processes that loop back; Comparison only for two different things.
7. How-to topics on a phone use PhoneScreen scenes, one action per item; button and menu names exactly as on screen, in English letters.
8. Every step, node, point, and phone action has a cue: one word copied from that scene's narration, in narration order.
9. If notes are given, base the script on them and add nothing that contradicts them.
10. Return only the JSON object.

Hindi (`language: hi-IN`) adds: write natural, simple spoken Hindi in Devanagari as an Indian teacher would explain it, keep common scientific terms in English where Indians say them in English (for example "water vapour"), and write on-screen labels in Hindi, except button and menu names, which stay in English letters as they appear on the phone.

Core rules in `revise.v1`: change only what the request asks; keep every untouched scene exactly the same, including its id; return the ids of changed scenes; at the video stage, prefer changes that keep narration the same when the request is only about

visuals, so no new audio is needed.

Each prompt gets a small test set in `scripts/eval-prompts.ts`: 10 English and 5 Hindi topics, checked for schema validity and the plan rules, run after any prompt change.

Database schema

Supabase Postgres holds the data; users come from Supabase Auth, and every other row belongs to one user. The Database Design doc is the full reference: it adds the llm_calls and worker_heartbeat tables, a frozen plan copy on each job, and the SQL for every table, policy, and function. The summary below shows the four core tables.

Table	Columns	Notes
`projects`	id (uuid), user_id, title, language (`en-IN`, `hi-IN`), input_mode (`idea`, `notes`, `brainstorm`), input_text, scene_plan (jsonb), plan_version (int), stage (`planning`, `storyboard`, `rendering`, `ready`), created_at, updated_at	`plan_version` goes up on every save; an autosave with an older version is rejected, so two tabs cannot overwrite each other
`messages`	id, project_id, user_id, stage (`brainstorm`, `storyboard`, `video`), role (`user`, `assistant`), content, created_at	Chat history for brainstorming and revision requests
`jobs`	id, project_id, user_id, plan_version, status, progress (0–100), changed_scene_ids (text[]), error, attempts, metrics (jsonb: characters voiced, render seconds, file size), claimed_at, started_at, finished_at	One row per render; `plan_version` records exactly which plan was rendered
`videos`	id, project_id, user_id, job_id, r2_key, size_bytes, duration_seconds, expires_at, deleted_at	One row per finished MP4; `expires_at` is 11:59 PM IST of the render day

Row-level security (RLS) is on for all four tables with one policy each: a signed-in user can read and write only rows where `user_id = auth.uid()`. The worker uses the service role key, which bypasses RLS, so that key exists only on the laptop.

The worker claims a job with one atomic update, so a job can never be picked up twice:

```
update jobs set status = 'voicing', claimed_at = now(), attempts = attempts +
1
where id = (select id from jobs where status = 'queued'
            order by created_at limit 1 for update skip locked)
returning *;
```

This runs as a Postgres function, `claim_next_job()`, called from the worker.

Web app pages and API routes

The web app has five pages and six API routes; anything that needs a secret key runs in an API route on the server, never in the browser.

Page	Purpose
`/login`	Google or email magic-link sign-in through Supabase Auth
`/projects`	Library: project cards with stage, language, and video status (ready, rendering, expires tonight, expired)
`/projects/new`	Planning: choose idea, notes, or brainstorm, and English or Hindi
`/projects/[id]`	Script and storyboard review: one card per scene (narration left, live template preview right), a revision chat box, and the render button
`/projects/[id]/video`	Preview player, live job progress, video-stage revision chat, download button, and the deletion time

The storyboard preview uses `@remotion/player` with the same templates the worker renders, so what the user reviews is exactly what gets rendered.

Route	Method	Does
`/api/brainstorm`	POST	Sends chat history to `brainstorm.v1`; saves both messages

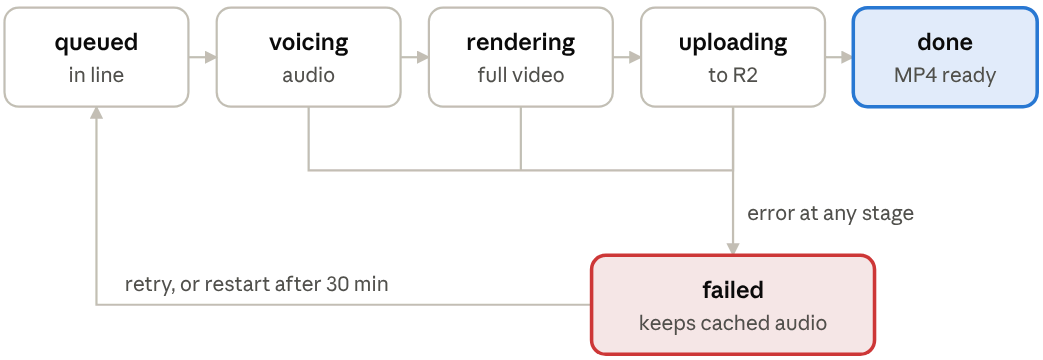
Route	Method	Does
`/api/script`	POST	Runs `script.v1` for a project, validates, saves the plan
`/api/revise`	POST	Runs `revise.v1` with the current plan and request; returns the new plan and changed scene ids
`/api/render`	POST	Validates the plan, checks voice quota, creates a `queued` job
`/api/jobs/[id]/retry`	POST	Puts a failed job back to `queued`, keeping completed stages
`/api/videos/[id]/download`	GET	Returns a 15-minute signed R2 link if the video has not expired

Autosave sends edits to `PATCH /api/projects/:id` 1 second after the user stops typing, with the current `plan_version`; the route validates the plan and saves it through the `save_plan()` database function. Job progress reaches the browser through a Supabase Realtime subscription on the user's `jobs` rows. The API Design doc lists every route.

Job pipeline

A render job moves through three working statuses in order (voicing, rendering, uploading) and saves its output after each one, so a failed job restarts at the stage that broke, not from the beginning.

A failed job retries from its last finished stage



Render job statuses · 5 stages, 1 failure state, retry

Scripting is not part of the job: it happens interactively in the web app before the user clicks Render. A job is created only for an approved plan, and records that plan's `plan_version`, so later edits never change a render already in progress.

Render worker

The worker is a single Node process on the laptop that claims a queued job every 5 seconds and handles one job at a time. It reuses the prototype's `voiceStep()` and `renderStep()` code from `scripts/steps.ts`, moved into `apps/worker`.

Audio cache. Each scene's audio is keyed by one hash, so unchanged scenes are never voiced twice:

Hash	Built from	Changes when
`audioHash`	voice provider, voice id, speaking speed, language, narration	The spoken words, voice, or speed change

Per-scene video clips and stitching were dropped after the prototype: a full 60–90 s render took 2 min 38 s to 3 min 52 s, fast enough that clip caching is not worth its complexity.

The three stages of a job:

1. **Voicing.** For each scene, reuse `audio\{audioHash}.mp3` if it exists; otherwise check credits, call the VoiceProvider, and save the MP3 and word timings. A visual-only revision never calls ElevenLabs.
2. **Rendering.** Build the timeline (`buildTimeline()`), copy the audio and music into Remotion's public folder, bundle, and render the whole video with `renderMedia()` : H.264, CRF 20, x264 preset `veryfast` , 2 parallel browser tabs. Output: `out\{jobId}.mp4` .
3. **Uploading.** Send the file to R2, write a `videos` row with its expiry time, delete the local file, and mark the job `done` .

Timing and captions. `buildTimeline()` turns audio lengths and word timings into frames before rendering. Items appear 0.2 s before their cue word is spoken, at least 0.6 s apart, and the first item of every scene within 0.4 s. Captions show up to 4 words at a time, breaking early at pauses over 0.3 s, with the current word highlighted. When `captionOverride` is set, its words are spread across the same spoken time span.

Resource safety. Rendering uses about 2–3 GB of RAM at 2 parallel tabs on the 4-core laptop; a crash retries once with 1 tab. Progress is written to the job row every 5%, so the web app shows steady movement.

Storage, retention, and cleanup

Finished videos live in one private R2 bucket for the rest of the render day; everything else stays on the laptop.

What	Where	Kept for
Finished MP4	R2 bucket `explainit-videos` , key `projects/{userId}/{projectId}/{jobId}.mp4`	Until 11:59 PM IST on the render day
Voice audio and timings	`D:\ExplainIt\cache\{projectId}\audio` and `timing`	As long as the project exists (reused for re-renders)
Local final MP4	`D:\ExplainIt\out`	Deleted once the upload succeeds
Render inputs	`packages/video/public/runs`	Replaced at the start of every render

The bucket is private: users download through a 15-minute signed link created by `/api/videos/[id]/download`, so video links cannot be shared or guessed.

Deletion runs three ways, so a video never outlives its day for long:

1. **Nightly job.** `node-cron` in the worker runs at 23:59 Asia/Kolkata, deletes R2 objects whose `expires_at` has passed, and sets `deleted_at` on their `videos` rows.
2. **Startup catch-up.** The same cleanup runs every time the worker starts, covering nights when the laptop was off.
3. **R2 lifecycle rule.** The bucket deletes any object older than 1 day, a safety net that needs no laptop at all.

Once a video row has `deleted_at`, the library shows "Video expired, render again". Because audio is cached, a re-render costs no ElevenLabs credits.

Config, secrets, and security

All settings live in one `.env` file at the project root, loaded by both apps; `.gitignore` excludes it, and `.env.example` lists the names with empty values.

Variable	Used by	Needed from
`GEMINI_API_KEY`	web, scripts	Done
`GEMINI_MODEL`, `GEMINI_FALLBACK_MODEL` (`gemini-flash-latest`, `gemini-flash-lite-latest`; pin exact IDs before launch)	web, scripts	Done
`GEMINI_THINKING` (`MINIMAL`)	web, scripts	Done
`ELEVENLABS_API_KEY`, `ELEVENLABS_VOICE_EN`, `ELEVENLABS_VOICE_HI`, `ELEVENLABS_MODEL`	worker, scripts	Done (voices still to choose)
`TTS_PROVIDER` (`edge` while building, `elevenlabs` for final videos)	worker, scripts	Done
`EDGE_RATE` (`+10%`), `EDGE_VOICE_EN`, `EDGE_VOICE_HI`, `PYTHON_BIN`	worker, scripts	Done
`OLLAMA_URL`, `OLLAMA_MODEL`, `OLLAMA_KEEP_ALIVE`	scripts (manual offline tests)	Done

Variable	Used by	Needed from
`MEDIA_DIR` (`D:\ExplainIt`)	worker, scripts	Done
`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`	web	M4
`SUPABASE_SERVICE_ROLE_KEY`	worker only	M5
`R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_BUCKET`	web (signed links), worker (upload, delete)	M5

Security rules:

- Only variables starting with `NEXT_PUBLIC_` reach the browser; the anon key is safe there because RLS limits it to the user's own rows.
- The service role key and ElevenLabs key never leave the laptop.
- R2 keys are scoped to the one bucket, with object read, write, and delete only.
- API routes check the Supabase session and that the project belongs to the user before any LLM or render call.
- Each user is limited to 20 LLM calls per hour and 5 render jobs per day, counted from the `messages` and `jobs` tables, to protect free quotas.
- Pasted notes are sent to Gemini's free tier, so the planning screen shows a one-line notice not to paste private material.

Error handling and quotas

Every known failure has a defined response and a plain-language message for the user; nothing fails silently.

Failure	Detected by	Response	User sees
LLM returns invalid JSON or breaks a plan rule	Zod and `checkPlanRules()`	Retry up to 2 times, sending the validation errors back to the model	"Fixing the script format..." then the script, or "Couldn't create a script, try rephrasing"

Failure	Detected by	Response	User sees
Gemini rate limit (429) or outage	HTTP status	Wait 5 s and retry once, then switch to Flash-Lite	Nothing, unless every option fails
ElevenLabs quota too low for the job	`remainingQuota()` before voicing	Stop before spending credits; offer edge-tts for this render	"Not enough voice credits this month. Use the free backup voice?"
ElevenLabs error mid-job	HTTP status	Retry the scene twice; finished scenes stay cached	Job fails at voicing with a Retry button
Remotion render crash or out of memory	Exception in `renderMedia()`	Retry the render once with 1 browser tab instead of 2	Job fails at rendering with a Retry button
Worker stopped mid-job (laptop closed)	Job stuck with `claimed_at` older than 30 minutes	On startup, requeue it; cached stages are skipped	Job resumes when the laptop is back
R2 upload fails	SDK error	Retry 3 times with backoff; keep the local file	Job fails at uploading with a Retry button
Autosave conflict	Older `plan_version`	Reject the write and reload the newer plan	"This project changed in another tab, reloaded"

Quotas we track: ElevenLabs characters (from its API before each job), Gemini requests (per-user hourly limit plus 429 handling), R2 storage (alert in the worker log above 5 GB, half the free 10 GB). Each job logs characters voiced, render seconds, and file size to the `jobs` row, so real costs are visible after M3.

Build order and testing

The command-line pipeline (M1–M3) is built and working; the remaining milestones turn it into the web product, each ending with a test you can run yourself.

Milestone	Builds	Status, or done when
M1: Script generator	`packages/shared` schema, rules, prompts, Gemini and Ollama adapters, repair loop, `npm run plan`	Done
M2: Voice and timings	edge-tts and ElevenLabs adapters, audio cache, `npm run voice`, `compare-voices`	Done
M3: Rendered video	`packages/video` with 7 templates, focus mode, motion, captions, fonts, `npm run render` and `make-video`	Done
M3.5: Evaluation	Choose the ElevenLabs voice, add music, score the 3 evaluation explainers, settle the English word range	Next: 2 of 3 videos pass "Would I post this?"
M4: Web app	`apps/web` pages, Supabase Auth, `projects` and `messages` tables, brainstorm, script, and revise routes, storyboard preview with `@remotion/player` using `packages/video`	Sign in, plan with all 3 modes, edit and revise a storyboard, reload with nothing lost
M5: Background jobs	`jobs` and `videos` tables, `claim_next_job()`, `apps/worker` built from `scripts/steps.ts`, Realtime progress, R2 upload, signed downloads	Click Render, watch progress, download the MP4; stop the worker mid-job and it resumes
M6: Polish	Video-stage revisions that re-voice only changed scenes, nightly cleanup and lifecycle rule, rate limits, error messages, library states, pinned Gemini model IDs, optional Vercel deploy with a hosted LLM fallback	20-topic test passes the PRD success metrics

Testing approach:

- **Unit tests (Vitest)** for the schema, plan rules, hashing, caption timing, and the expiry time calculation.
- **Prompt evals** with `scripts/eval-prompts.ts` after every prompt change.
- **Visual checks** of each template in Remotion Studio before full renders.
- **A fixed test set** of 20 topics (15 English, 5 Hindi) reused at M3 and M6 so results can be compared.

Decisions on open questions

The prototype settled most open questions with measurements; two remain for the evaluation.

Question	Decision
Word range for 60–90 s	Decided: the user picks 60 s or 90 s. 60 s targets 125–155 words and 90 s targets 185–220 words, based on the measured pace of about 135–145 words per minute (Hindi 171 words gave 74 s; English 196 words gave 90 s). Stored as projects.duration_target.
ElevenLabs model	Multilingual v2 for both languages, about 1,000–1,400 credits per video (7–10 videos a month free); Flash v2.5 if credits run short. Develop with edge-tts and spend credits only on final videos.
ElevenLabs voices	Open until M3.5: compare 2–3 candidates with `npm run compare-voices`, preferring one voice for both languages.
Per-scene rendering plus stitching	Dropped. Full renders take 2–4 minutes; the audio cache keeps revisions cheap.
LLM fallback	Gemini Flash-Lite only. Ollama was too slow on the laptop CPU (about 135 s per call) and stays a manual offline option.
Gemini speed	`GEMINI_THINKING=MINIMAL`: about 30–40 s per script, versus 263 s by default. The web app shows progress messages instead of promising seconds.

Question	Decision
Deployment	Keep the web app on the laptop through testing; deploy to Vercel later with a hosted LLM fallback.
Background music	Decided: no background music in V1. The pipeline already renders without music when the music folder is empty.